

INTERNSHIP PROJECT REPORT

Secure MERN Exam System with Blockchain Verification

A Cryptographically Sealed, AI-Proctored Online
Examination Platform

Prepared By: [Your Name]

Mentor: [Mentor Name & Designation]

Department: Computer Science &
Engineering

Academic Supervisor: [Supervisor Name]

Date: May 24, 2026

Institution: [Your University Name]

Acknowledgment

I would like to express my gratitude to my industry mentor and academic supervisor for their guidance during this internship. Their advice on blockchain architecture, computer vision models, and secure coding practices helped shape this project.

I also thank the department and university for providing the resources to implement and test this system. This experience has deepened my understanding of decentralized verification and proctoring architectures.

[Your Name]

May 2026

Abstract

Online testing systems require security measures to ensure integrity. Traditional online exam portals are vulnerable to client-side cheating and database tampering.

This report details the design and implementation of a **Secure MERN Exam System with Blockchain Verification**. The system combines:

- **AI-Powered Proctoring:** Runs client-side TensorFlow.js models in the student's browser to analyze face mesh, gaze direction, hand gestures, and check for banned objects like mobile phones.
- **WAF and Zero-Trust Access:** Filters request payloads for SQL/NoSQL injections and monitors access logs, suspending database keys if high-risk actions are detected.
- **Self-Healing Database Guardian:** A polling daemon that automatically reverts unauthorized changes to student scores.
- **Decentralized Sealing:** Anchors exam results and database roots to an Ethereum blockchain via the `CredentialSeal.sol` smart contract.

This multi-layered approach ensures exam results are verifiable and secure against client and database tampering.

Table of Contents

1. Introduction	1
2. Requirement Analysis & Specifications	2
3. System Design & Architecture	3
4. Core Modules & Implementation Details	4
5. Visual Demonstration & Screens	6
6. Testing, Auditing & Evaluation	8
7. Conclusion & Future Scope	9
8. References	10

1. Introduction

1.1 Project Context & Motivation

As online assessments become standard, academic dishonesty remains a challenge. Remote exams are vulnerable to cheating, such as tab switching, secondary devices, copy-paste bypasses, and screen captures.

Additionally, database vulnerabilities can allow attackers to modify scores directly in the database. This project implements client-side proctoring, server-side firewall validation, and blockchain verification to address these issues.

1.2 System Objectives

- **Establish Identity Trust:** Verify user presence via continuous face mesh tracking.
- **Restrict Client Environment:** Block keyboard shortcuts, PrintScreen, devtools access, and tab switching.
- **Secure the Storage Layer:** Encrypt database records using rotating keys.
- **Automate Tamper Recovery:** Revert unauthorized database changes.
- **Seal Credentials:** Store result hashes on-chain to prevent credential forgery.

2. Requirement Analysis & Specifications

The system requires functional separation of user roles and security controls.

2.1 Functional Requirements

- **Role-Based Access:** Different interfaces for Students, Teachers, School Admins, and Super Admins.
- **Exam Creator:** Enable teachers to create tests with random question sequencing.
- **Automated Proctoring:** Monitor candidate streams for violations.
- **Verification Portal:** Verify result hashes on-chain.

2.2 Non-Functional Requirements

- **Low Performance Overhead:** Perform computer vision checks on the client to reduce server load.
- **High Availability:** Enable offline verification of certificates using signatures.
- **Key Management:** Rotate keys without database downtime.

2.3 Technology Stack

Component	Technology Used	Description
Frontend	HTML5, custom CSS, Vanilla JavaScript	Sleek dashboards and WebRTC camera captures
AI Engine	TensorFlow.js (CocoSSD, FaceMesh)	Real-time client-side proctoring
Backend	Node.js, Express	REST API endpoints and Socket.io controls
Database	MongoDB, Mongoose	Schema models and session persistence
Blockchain	Hardhat, Solidity, Ethers.js	Smart contracts for result sealing

3. System Design & Architecture

The system employs a multi-tiered architecture that separates presentation, security filtering, business logic, and verification.

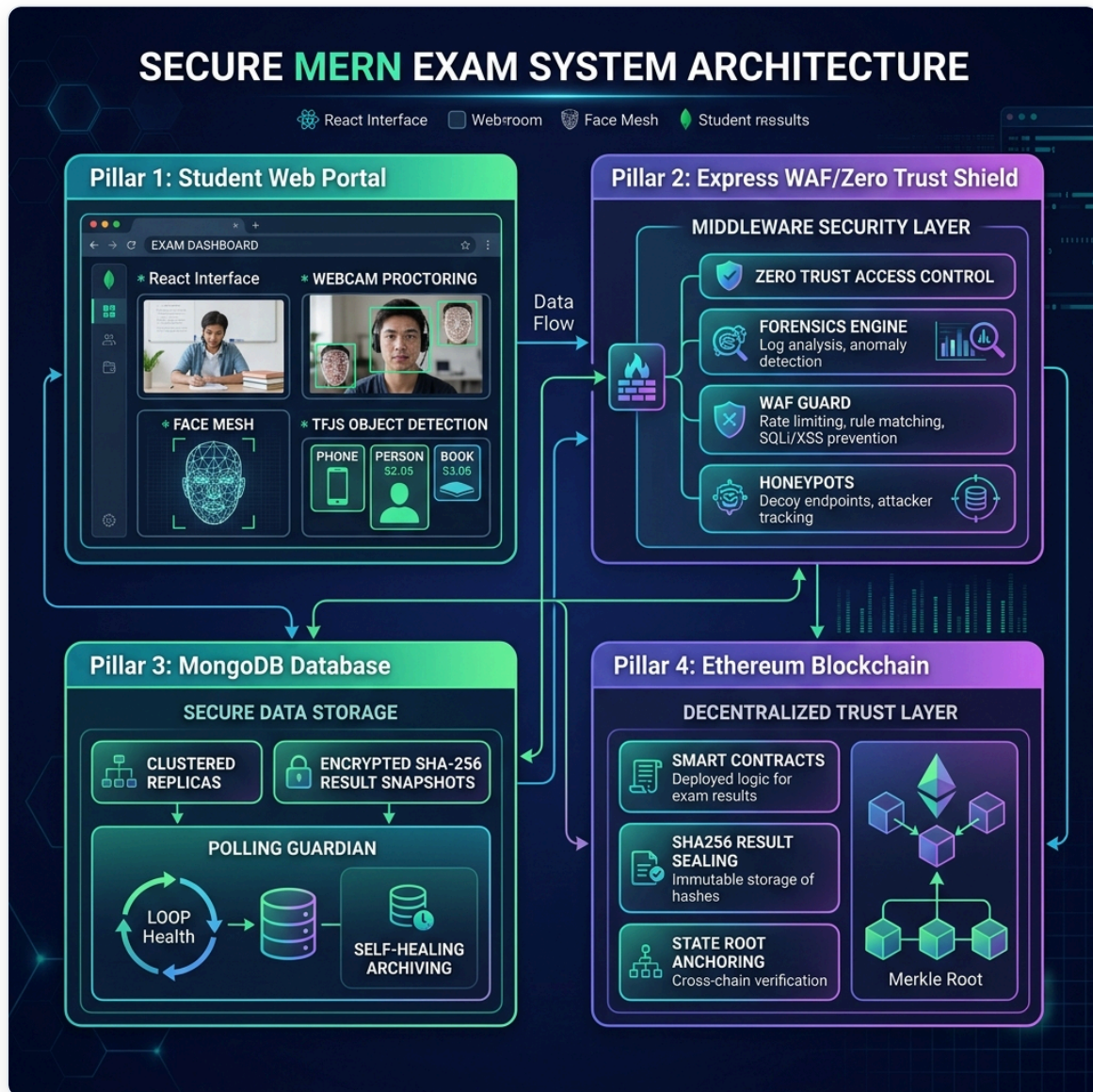


Figure 3.1: Architecture Diagram showing Frontend, WAF/Zero Trust, Database, and Ethereum blockchain.

3.1 Architectural Workflow

When a student begins an exam, the browser requests a session token. The request passes through the WAF and Zero-Trust middleware. If approved, a session is initiated and camera streams are routed to the proctoring engine.

Upon exam completion, the server hashes the result payload and saves it. Ethers.js is used to write this hash to the blockchain ledger.

4. Core Modules & Implementation Details

4.1 Client-Side AI Proctoring Engine

Located in `proctor.js`, this module uses TensorFlow.js to process webcam feeds in the browser.

```
// Object Detection Loop snippet
const objects = await this.objectModel.detect(video);
objects.forEach(obj => {
  if (obj.class === 'cell phone' && obj.score > 0.65) {
    this._handleViolation('mobile-critical', 'Phone detected.');
```

It monitors for several violations:

- **Face Mesh Tracking:** If a student's gaze drifts from the screen or no face is detected, a violation is recorded.
- **Banned Objects:** Detects books, laptops, and mobile phones.
- **Browser Restriction:** Intercepts right-click context menus, PrintScreen, and DevTools access.

4.2 Server-Side Security Firewall (WAF)

The WAF filters request payloads for injection patterns:

```
const NOSQL_INJECTION_PATTERN = /(\$ne|\$gt|\$lt|\$gte|\$lte|\$regex|\$where)/i;
const SQL_INJECTION_PATTERN = /('s*or\s*|--|select\s+.*\s+from)/i;
```

Requests containing matching patterns are blocked and logged.

4.3 Database Self-Healing Guardian

The database guardian operates as a polling service:

- **Snapshot Verification:** Scans database results every 60 seconds and compares them against snapshots.
- **Automated Recovery:** Reverts unauthorized changes to scores or answers.

5. Visual Demonstration & Screens

Below are captures of the user interfaces and proctoring feeds from the system.



Figure 5.1: Proctoring screen showing face mesh tracking and real-time alerts.

5.1 User Authentication Portals

The system provides role-specific login interfaces for students, teachers, and administrators.



Figure 5.2: Student Login Portal



Figure 5.3: Teacher Login Portal

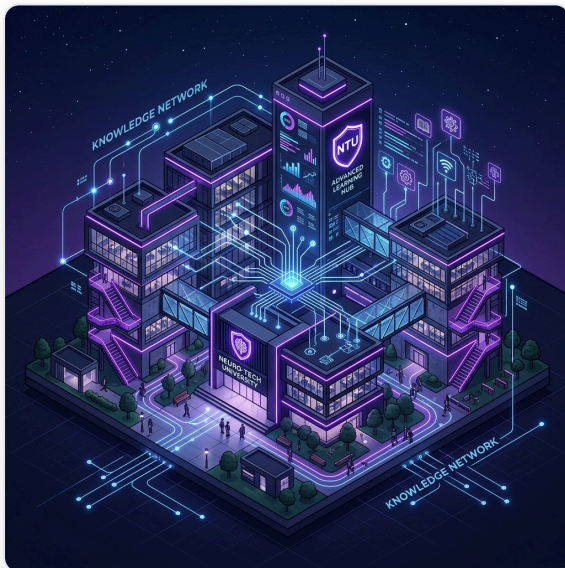


Figure 5.4: School Admin Login Portal



Figure 5.5: Super Admin Login Portal

6. Testing, Auditing & Evaluation

6.1 Security Intrusion Testing

The WAF middleware was tested with SQL injection attacks (e.g., `' OR 1=1 --`). The requests were blocked, and a high-severity security event was logged.

Accessing the decoy endpoint `/api/system/backup-config` resulted in immediate lockout and user suspension.

6.2 Self-Healing Verification

To test database auditing, a score in MongoDB was modified from 85 to 100.

Tamper Alert: The system detected the discrepancy, restored the score to 85, and logged the incident within the 60-second polling window.

7. Conclusion & Future Scope

7.1 Key Achievements

- Implemented client-side proctoring that checks for violations in the browser.
- Configured WAF and Zero-Trust gateways to filter traffic and handle keys.
- Created a database auditing system that detects and reverts score modification.
- Developed blockchain result sealing using Solidity smart contracts.

7.2 Future Scope

- **L2 Deployments:** Use Layer 2 blockchains (e.g., Optimism) to reduce transaction costs.
- **Facial Recognition:** Add biometrics to verify candidate identity during the exam.

8. References

1. MediaPipe Face Mesh Documentation, Google AI.
2. TensorFlow.js Model Repository (CocoSSD).
3. Solidity Smart Contract Development Guide, Ethereum Foundation.
4. NIST Zero Trust Architecture Guidelines (SP 800-207).